

CSE241 — Object-Oriented Computer Programming

Restaurant Management System

Submitted to:

Dr.Mahmoud Ibrahim

Eng.Andrew Ayman

Team Members:

Iujain mohamed helmi — 24P0285

Heba Hassan Badawy — 24P0481

Farha Melad Kamal— 24P0467

Habiba Hassan Mostafa — 24P0420

Malak Moustafa ali — 24P0239

1. Introduction

1.1 Problem Statement

Manual management of restaurant reservations and table ordering is error-prone, inefficient, and difficult to scale. Customers have no self-service option for browsing availability or making reservations, staff cannot easily track order progress, and administrators lack a centralised view of operations. This project addresses those challenges by providing a role-based desktop application that integrates reservation management, menu ordering, real-time status tracking, and administration into a single cohesive system.

1.2 Objectives

- Implement a three-role system — Customer, Admin, and Waiter — each with a tailored interface and appropriate privileges.
- Support the full reservation lifecycle: creation, validation, modification, and cancellation.
- Provide menu browsing with category filtering, customer ordering, and waiter-assisted ordering.
- Generate invoices with automatic tax and service charge calculation, and process balance-based payments.
- Demonstrate core OOP principles — encapsulation, inheritance, abstraction, and polymorphism — throughout the codebase.

1.3 Scope

The system is a single-tier JavaFX desktop application using an in-memory data store. It covers customer self-service (registration, reservation, ordering, payment), waiter workflows (table assignment, order entry, status updates), and administrative CRUD for tables, menu items, and categories.

2. System and Design Analysis

2.1 Architecture Overview

The application follows a single-tier architecture: all components — model, controller, data store, and networking — execute within a single JVM process. Three packages organise the code:

Package	Contents
projecttry1	All model classes, all FXML controllers, networking classes, App, Main, Session, Database
enums	8 enumerations covering order, payment, reservation, table states, and staff roles
gui.tasks	MenuLoadTask and TableAvailabilityService — JavaFX concurrent helpers

Application startup follows this sequence: Main delegates to App.start(), which initialises the in-memory Database, spawns a background TCP server thread on port 8080, and loads the Login screen. Navigation is handled entirely by App.switchScene(), a static utility that loads any FXML file, attaches the global stylesheet, and replaces the primary Stage's scene. Session persists the logged-in user across all scene transitions.

2.2 Data Model

All state is held in public static ArrayList fields in Database.java. The class also exposes synchronised accessor methods that return defensive copies (new ArrayList<>(...)) for thread-safe reads.

3. OOP Concepts and Design Decisions

3.1 Encapsulation

Every model class uses private fields exposed through public getters and setters. Validation is embedded in the mutators rather than scattered across controllers:

- `Customer.setPassword()` and `Customer.setPhoneNumber()` invoke `validatePassword()` (minimum 6 characters) and `validatePhoneNumber()` (exactly 11 digits) before accepting a new value.
- `Table` constructor throws `IllegalArgumentException` if `capacity ≤ 0`; `setCapacity()` silently rejects non-positive values.
- `MenuItem` constructor and `setName()` throw `IllegalArgumentException` on a blank name; `setPrice()` silently rejects negative values.
- `OrderItem` constructor and `setQuantity()` throw `IllegalArgumentException` for non-positive quantities.
- `serviceCharge`, and `totalAmount` are intentionally commented out — these are read-only `calculateInvoice()` — enforcing immutability of the financial calculation.
- `Order.getItems()` returns new `ArrayList<>(items)` rather than the internal list, preventing external modification of the order's line items.

3.2 Inheritance

The staff hierarchy is built on an abstract base class:

Class	Type	Adds Over Parent
Staff	Abstract	username, password, role (Role enum), workingHours, dateOfBirth
Admin extends Staff	Concrete	Table CRUD, MenuItem CRUD, MenuCategory CRUD — all delegating to Database
Waiter extends Staff	Concrete	assignedTables list, takeOrder(), updateOrderStatus() (with TCP push), manageSeating(), checkOutTable()

`Customer` is a standalone class — it does not extend `Staff` — reflecting the semantic distinction between system staff and registered customers.

3.3 Abstraction

- Staff is declared abstract and cannot be instantiated directly. All staff objects are either Admin or Waiter instances.

3.4 Polymorphism

- instanceof with downcast: Several controllers receive a Staff object from Session and check its concrete type before calling role-specific methods. For example, WaiterDashboardController checks if (staff instanceof Waiter) before calling waiter.updateOrderStatus(). The same pattern appears in AdminTableManageController, AdminMenuManageController, AdminCategoryManageController, and WaiterTakeOrderController.
- @Override toString(): All model classes — Customer, Order, OrderItem, MenuItem, MenuCategory, Invoice, Table, Reservation — override toString() to provide meaningful string representations.
- JavaFX generic subclassing: MenuLoadTask extends Task<List<MenuItem>> overriding call(); TableAvailabilityService extends ScheduledService<List<Table>> overriding createTask(). This demonstrates polymorphism through generic type specialisation.

3.5 Enumerations

Eight enums define all discrete state values, eliminating magic strings and enabling compile-time safety:

Enum	Values	Purpose
OrderStatus	PLACED, PREPARING, SERVED, PAID	Order lifecycle; forward-only progression enforced by Order.updateStatus()
PaymentMethod	CASH, CREDIT_CARD, ONLINE	Payment method selection at checkout
PaymentStatus	PENDING, PAID	Invoice payment state
ReservationStatus	ACTIVE, CANCELLED, COMPLETED	Reservation lifecycle
Role	ADMIN, WAITER	Staff role differentiation for login routing
TableStatus	AVAILABLE, RESERVED, OCCUPIED	Current table state
TableType	INDOOR, OUTDOOR, VIP	Table classification used in filtering

4. Implementation

4.1 Authentication

Three login paths serve distinct roles. Customer login (`LoginController`) iterates `Database.customers` for a case-insensitive username match, then calls `customer.login(username, password)` for plain-string comparison. On success, `Session.setCurrentCustomer()` is called and the Customer Dashboard is loaded. Staff login (`StaffLoginController`) follows the same pattern against `Database.staffMembers`, then reads the Role enum to route ADMIN users to the Admin Dashboard and WAITER users to the Waiter Dashboard.

Customer registration (`RegisterController`) validates the username for uniqueness, enforces password length (≥ 6 characters) and phone format (exactly 11 digits), and creates a new Customer with an initial balance of \$500. Date of birth is optional and defaults to 20 years before the current date if omitted. Passwords are stored as plain strings for simplicity.

4.2 Reservation System

The full reservation lifecycle is implemented through `Customer.makeReservation()`, `Reservation.updateReservation()`, and `Customer.cancelReservation()`. Validation is enforced at the model level: the `Reservation` constructor rejects a party size that exceeds table capacity or is non-positive, `isValidReservation()` requires a future date and a time between 10:00 and 23:00. Double-booking prevention iterates `Database.reservations` checking for an ACTIVE reservation on the same table at the same datetime (exact equality). On cancellation, the table status is restored to AVAILABLE.

The `MakeReservationController` presents a date picker and a time `ComboBox` with 30-minute steps in the valid range. Update and cancel operations are available from the same screen.

4.3 Menu and Ordering

The menu is pre-loaded with eight items across four categories (Appetizers, Main Course, Beverages, Desserts). `MenuController` loads items asynchronously using `MenuLoadTask` displays them in a `TableView` with a category filter `ComboBox`. Each "Add to Order" click adds one unit of the selected item to the customer's active (unpaid) order, or creates a new Order linked to the first available table as a fallback. The running total is shown in a label; `Order.addItem()` throws `IllegalStateException` if an item is marked unavailable.

Waiters use `WaiterTakeOrderController`, which provides a full draft builder: they select a table and optionally a customer (null for walk-ins), then choose items with a quantity `Spinner` (1–50) and an optional notes field. Adding the same item with the same notes increments the existing draft entry rather than creating a duplicate. The order is placed by calling `Waiter.takeOrder()` if the session staff is a `Waiter` instance, or by direct `Database` insertion otherwise.

4.4 Invoice and Payment

`Invoice.calculateInvoice()` computes all financial values automatically on construction. The formula is: $\text{subtotal} = \text{sum of } (\text{unit price} \times \text{quantity})$ for each `OrderItem`; $\text{taxAmount} = \text{subtotal} \times 0.10$; $\text{serviceCharge} = \text{subtotal} \times 0.25$; $\text{totalAmount} = \text{subtotal} + \text{taxAmount} + \text{serviceCharge}$. The setters for these fields are intentionally disabled to enforce read-only semantics.

`CheckoutController` locates the customer's latest unpaid `Order`, finds or creates its `Invoice`, and presents the breakdown. It validates that the customer's balance covers the total; if not, an error `Alert` is shown. On sufficient balance, `Customer.checkoutAndPay()` deducts the total and awards $\text{loyaltyPoints} += \text{floor}(\text{totalAmount} / 10)$. The invoice is marked `PAID` with the selected `PaymentMethod` (Cash, Credit Card, or Online), the order status is set to `PAID`.

4.5 Admin Functionality

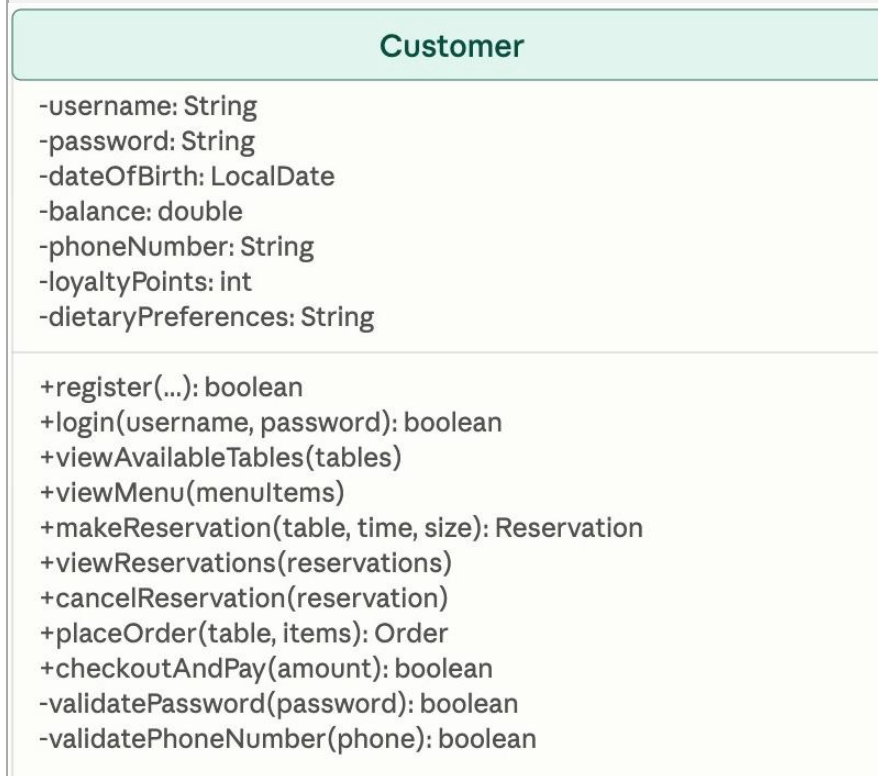
The Admin Dashboard displays statistics (customer, table, order, and reservation counts) alongside a live orders table. Four sub-screens provide CRUD for tables (`AdminTableManageController`), menu items (`AdminMenuManageController`), categories (`AdminCategoryManageController`), and read-only views for customers and reservations. Each CRUD controller validates input locally (non-empty names, positive capacity/price, duplicate detection) before delegating to the corresponding Admin method, which in turn operates on the Database collections.

4.6 Waiter Functionality

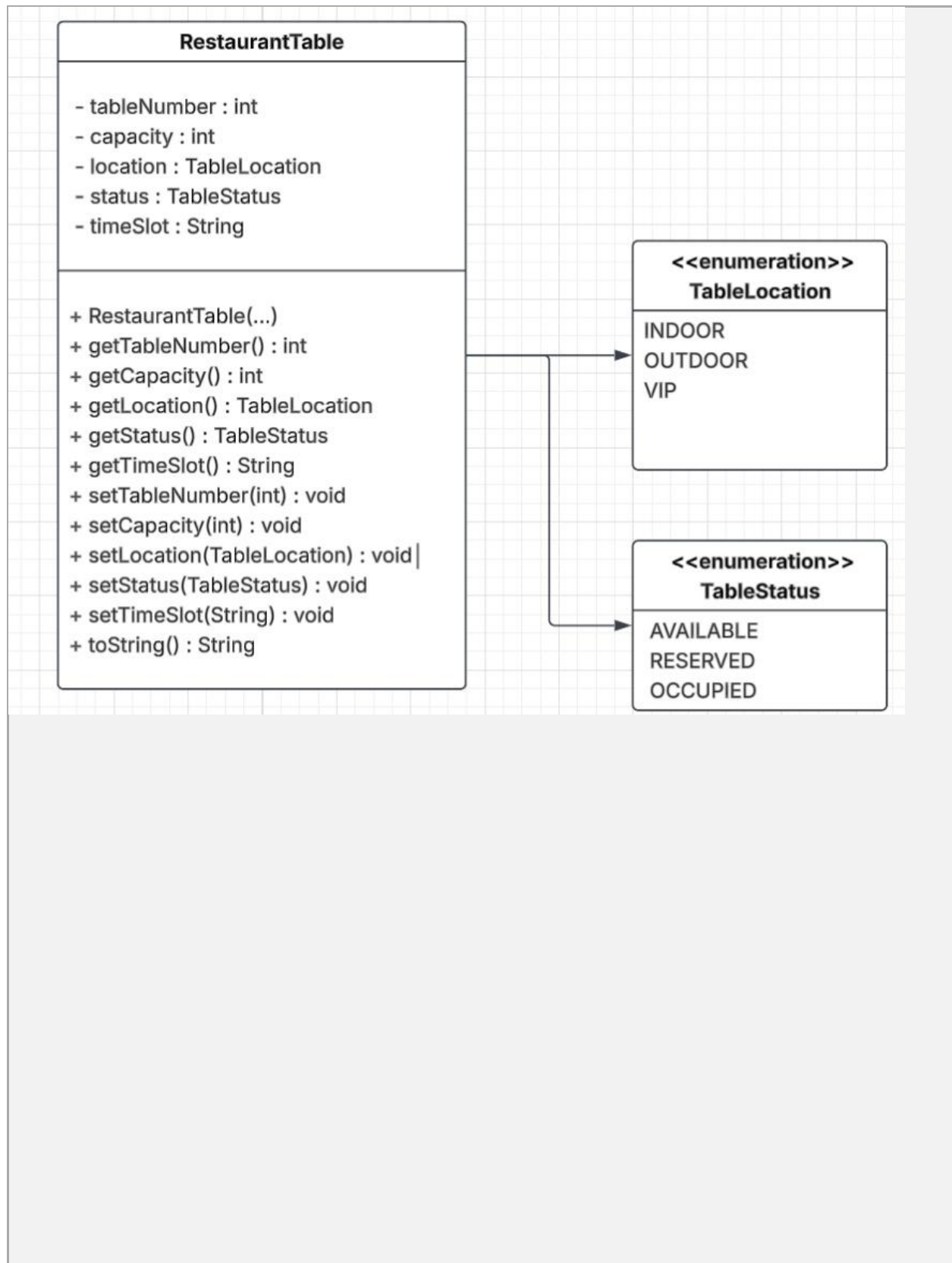
`WaiterDashboardController` displays all orders and allows status advancement via a `ComboBox`. `Order.updateStatus()` enforces a forward-only state machine using enum ordinal comparison — reverting from `SERVED` to `PREPARING`. When a waiter advances status, `Waiter.updateOrderStatus()` is called, which publishes the new status to the TCP server via `OrderStatusSocketPublisher` so the customer's dashboard updates in real time. Table assignment (`Waiter.manageSeating()`) marks the table `OCCUPIED`; unassignment (`Waiter.checkOutTable()`) restores it to `AVAILABLE`.

5. UML Class Diagram

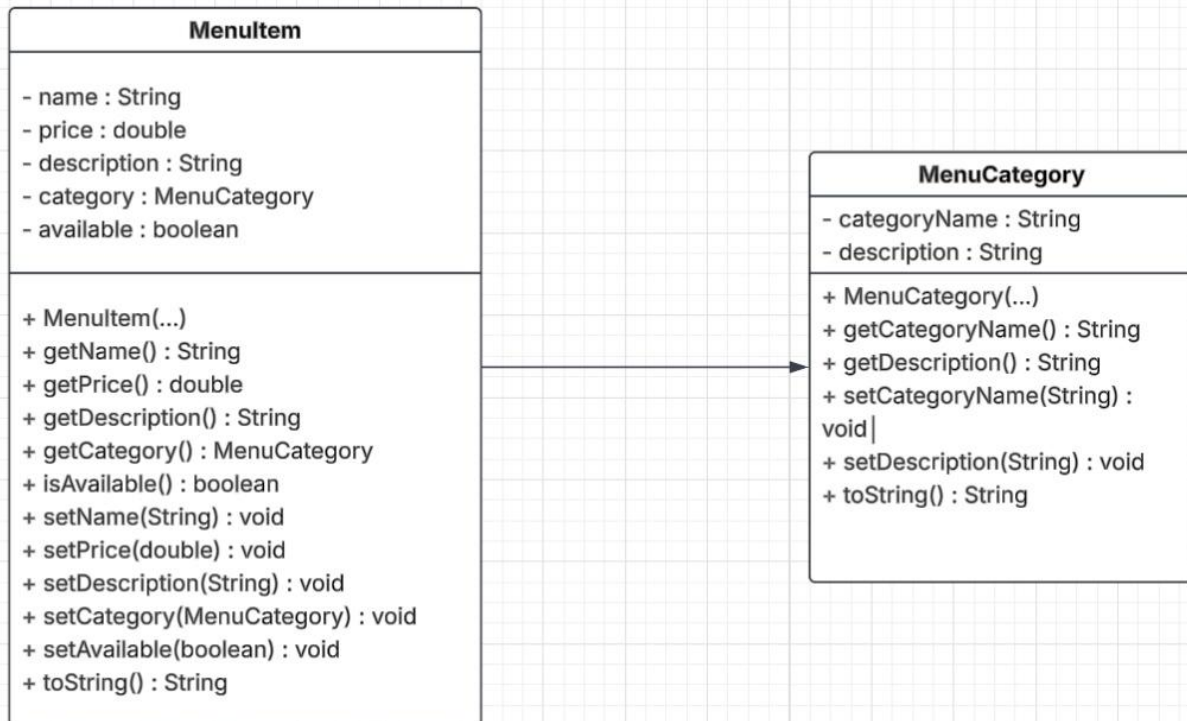
5.1-customer



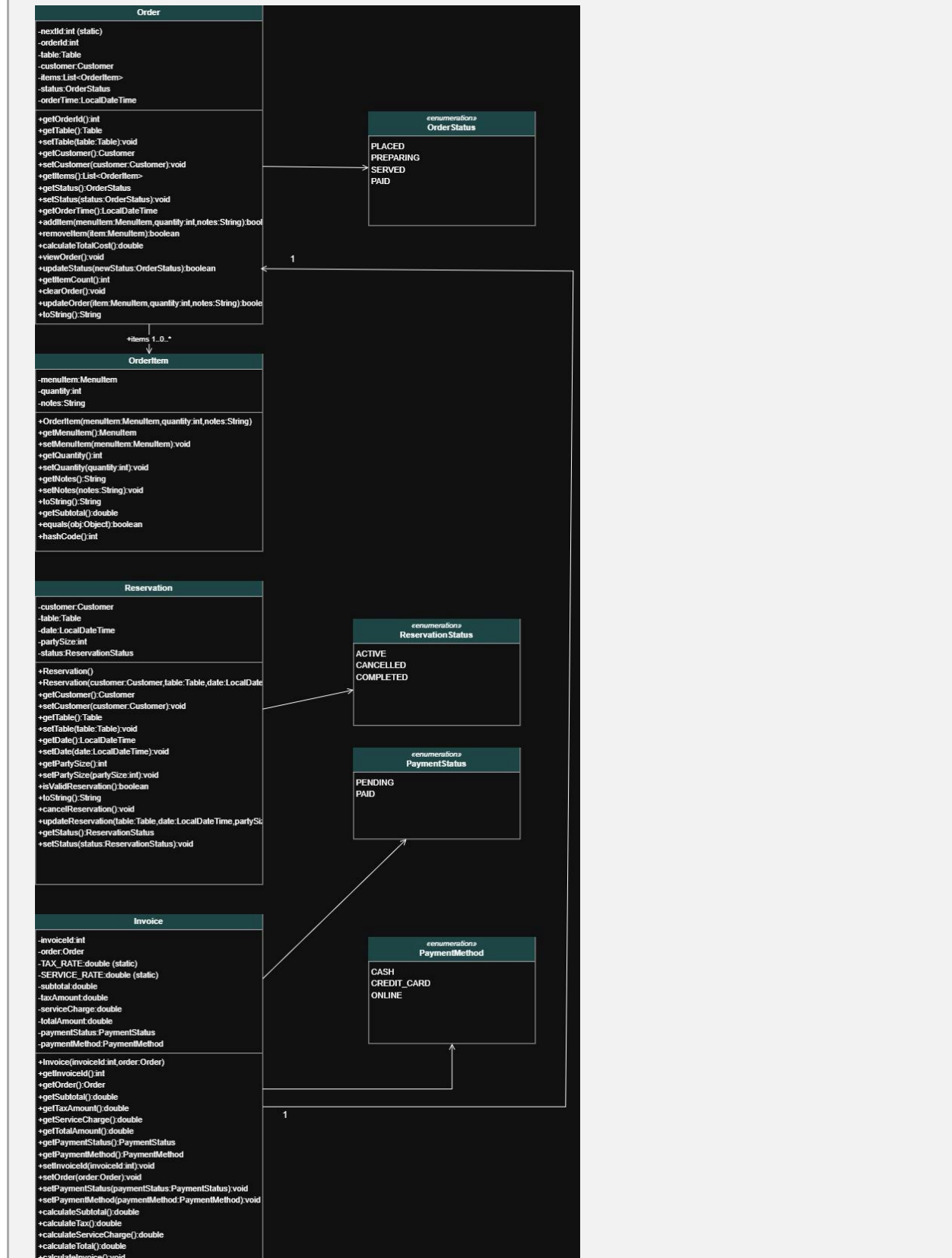
5.2- Table



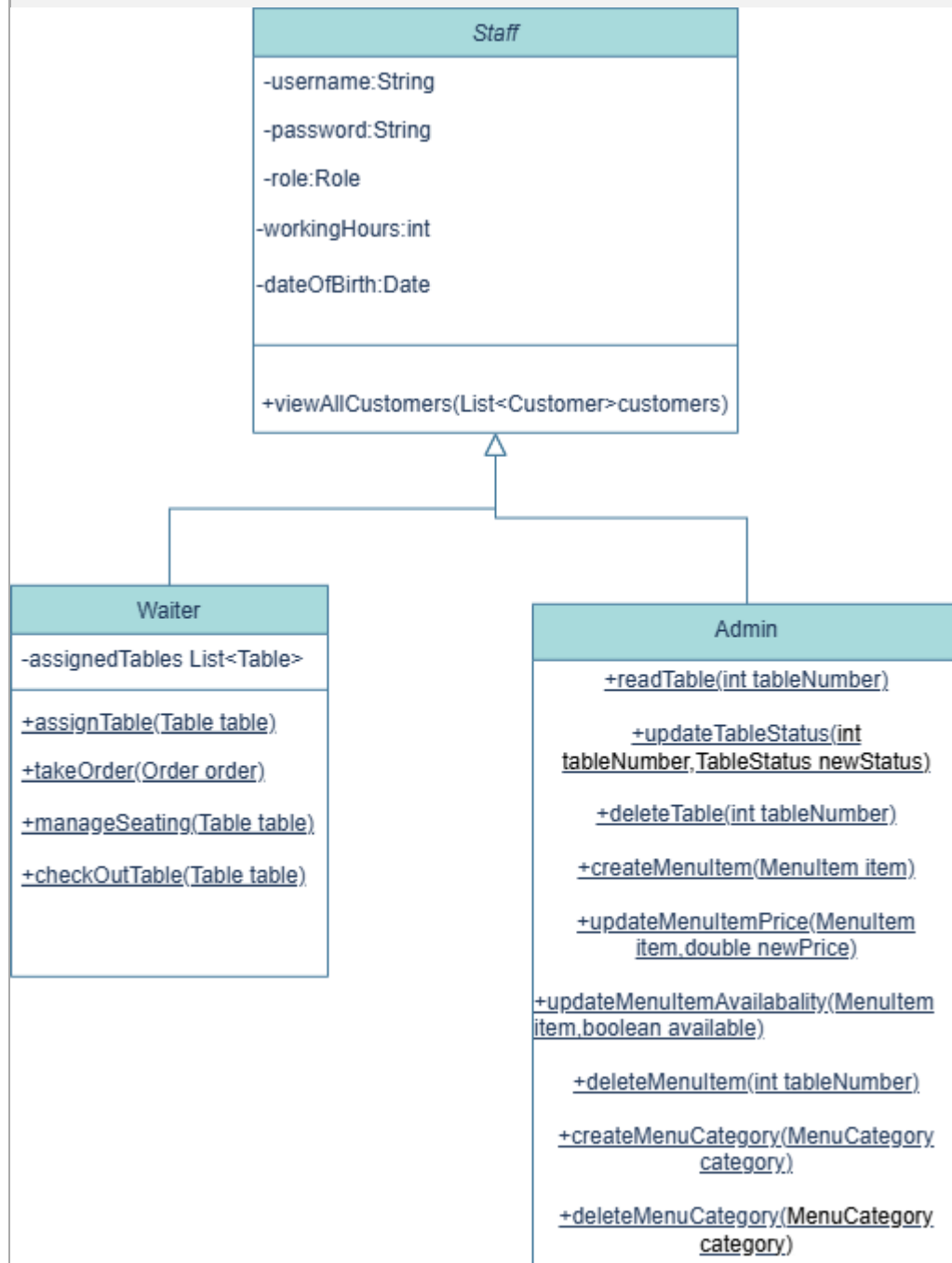
5.3-Menu



5.4- Order, Reservation, Invoice



5.5 – staff and admin



6. GUI

6.1 Screen Inventory

The application comprises FXML screens, each paired with a dedicated controller class. Navigation is handled exclusively by `App.switchScene()`, which loads a new FXML, applies the global stylesheet, and replaces the Stage scene no controller state is preserved between transitions.

Screen	FXML / Controller	Role	Purpose
Customer Login	Login / LoginController	Customer	Username + password authentication
Customer Register	Register / RegisterController	Customer	New account creation with validation
Staff Login	StaffLogin / StaffLoginController	Admin / Waiter	Role-based staff authentication
Customer Dashboard	CustDashboard / CustDashboardController	Customer	Profile, balance, loyalty points, order status tracker
Table View	TableView / TableController	Customer	Browse and filter tables; live availability polling
Make Reservation	MakeReservation / MakeReservationController	Customer	Create, update, or cancel a reservation
Menu View	MenuView / MenuController	Customer	Browse menu by category; add items to order
Checkout	Checkout / CheckoutController	Customer	Invoice breakdown and payment
Admin Dashboard	AdminDashboard / AdminDashboardController	Admin	Statistics overview and live orders table
Admin Manage Tables	AdminTableManage / AdminTableManageController	Admin	Full CRUD for tables
Admin Manage Menu	AdminMenuManage / AdminMenuManageController	Admin	Full CRUD for menu items
Admin Manage Categories	AdminCategoryManage / AdminCategoryManageController	Admin	CRUD for menu categories
Admin View Customers	AdminViewCustomers / AdminViewCustomersController	Admin	Read-only customer list with aggregate stats
Admin View Reservations	AdminViewReservations / AdminViewReservationsController	Admin	Read-only reservation list

Screen	FXML / Controller	Role	Purpose
Waiter Dashboard	WaiterDashboard / WaiterDashboardController	Waiter	Order status management, table assignment
Waiter Take Order	WaiterTakeOrder / WaiterTakeOrderController	Waiter	Draft-based order builder

6.2 Styling

A single global stylesheet (style.css, 119 lines) is applied to every scene. The colour scheme uses an orange accent (#e67e22) for primary buttons and highlights, a dark header (#34495e) for table column headers, and white card-style containers. Text inputs, ComboBoxes, and DatePickers receive focus-ring effects. TableView rows are styled with alternating backgrounds and a hover highlight. The UI font is Segoe UI / Arial.

6.3 User Feedback

All controllers provide immediate feedback through two mechanisms. Inline feedback labels change colour — green for success, red for errors — without requiring a dialog. For significant events (payment success, reservation confirmation, cancellation confirmation, payment failure) the application uses `Alert.AlertType.INFORMATION`, `CONFIRMATION`, or `ERROR` dialogs, giving the user an explicit acknowledgement step before the application proceeds.

6.4 Sample GUI Screenshots

Restaurant Management & Order System



Restaurant Dashboard



Heba

★ Loyalty: 0 pts

|

 Balance: \$1200.00

 Active Order Status

No Active Order

1. PLACED

→

2. PREPARING

→

3. SERVED

Status: No active order placed yet.

 View & Order Menu

 Browse Tables

 Make a Reservation

 Checkout / Pay Invoice

Logout

Restaurant Management & Order System



Create an Account

Username

Password

Date of Birth



Phone Number

Dietary Preferences

Register

[Already have an account? Log in](#)

Restaurant Management & Order System



Welcome Back

Username

Password

Log In

No account found with that username.

[Don't have an account? Register here](#)

[Staff? Login here](#)

Restaurant Management & Order System



Manage Reservation

Date



Time Slot



Party Size

Select Table



Could not create reservation: Invalid party size.

✓ Make New Reservation

Update Reservation

Cancel Reservation

Back to Dashboard

Figure 2: Customer



CHECKOUT

Insufficient Balance



Your balance is \$500.00 but the total is \$972.00.
Please top up your account before checking out.

OK

Service Charge (25%): **\$180.00**

Total Amount: \$972.00

Select Payment Method:



Cash



Credit Card



Online Payment

Payment Status: PENDING

[Back to Dashboard](#)

[Confirm Payment](#)

Page 22 of 29

7. Multithreading

The project implements six concurrent mechanisms, covering both the JavaFX concurrent API and raw Java threading. All UI updates originating from background threads are through `Platform.runLater()` or use JavaFX Task/ScheduledService callbacks, which are guaranteed to execute on the JavaFX Application Thread.

Thread / Mechanism	Where Started	Work Performed	Thread Safety
OrderServerThread (plain Thread)	App.start()	Runs <code>ServerSocket.accept()</code> loop on port 8080; never blocks UI	<code>ConcurrentHashMap</code> for client registry
Virtual Thread Pool (ExecutorService)	Server.start()	Each TCP client handled by a virtual thread via <code>Executors.newVirtualThreadPerTaskExecutor()</code>	<code>ConcurrentHashMap</code> ; <code>PrintWriter</code> auto-flush
OrderStatusClientThread (plain Thread)	CustDashboardController.initSocketClient()	Persistent socket read loop; sends REGISTER username on connect, reads ORDER_STATUS messages	<code>Platform.runLater()</code> for all UI updates
SocketPublisherThread (plain Thread)	OrderStatusSocketPublisher (static)	Short-lived socket open → PUBLISH message → close; fire-and-forget	No shared state; no UI access
MenuLoadThread (ExecutorService + Task)	MenuController.loadMenuAsync()	<code>MenuLoadTask.call()</code> returns <code>Database.getMenuItems()</code> (synchronised copy)	JavaFX Task; <code>setOnSucceeded</code> runs on FX thread
TableAvailabilityService (ScheduledService)	TableController.startAvailabilityPolling()	Polls <code>Database.getTables()</code> every 3 seconds via <code>TableAvailabilityService.createTask()</code>	<code>ScheduledService</code> ; <code>setOnSucceeded</code> runs on FX thread; cancelled in <code>handleBack()</code>

8. Networking

The application implements a local TCP publish-subscribe broker for real-time order-status notification. The server runs on port 8080 and maintains a registry of connected customer clients, allowing any waiter or checkout action to push a status update directly to the relevant customer's dashboard without polling.

8.1 Architecture

The networking layer consists of three classes and one functional interface:

- **Server:** Opens a `ServerSocket` on port 8080. Each accepted connection is dispatched to a virtual thread via `Executors.newVirtualThreadPerTaskExecutor()`. A `ConcurrentHashMap` maps authenticated usernames to their `PrintWriter`. `Server.start()` handles `BindException` gracefully (prints a warning and continues) so the application survives if the port is already in use.
- **OrderStatusClient (Runnable):** Opened once when a customer logs in. It registers with the server (`REGISTER|username`), then loops reading `ORDER_STATUS|orderId|status` lines. Each received message is dispatched to the registered `OrderStatusListener` lambda inside `Platform.runLater()`, updating the step indicator on the Customer Dashboard.
- **OrderStatusSocketPublisher:** A static utility that opens a short-lived `Socket`, sends a `PUBLISH` message, and closes. Called by `Waiter.updateOrderStatus()`, `MenuController.addToOrder()`, `CheckoutController.confirmPayment()`, and `WaiterTakeOrderController.handlePlaceOrder()` — always on a new `Thread` to avoid blocking the UI.
- **OrderStatusListener (@FunctionalInterface):** Defines the callback contract `onStatusUpdate(String orderId, String status)`. Implemented as a lambda when `CustDashboardController` initialises the client.

8.2 Message Protocol

Direction	Message Format	Meaning
Client → Server	<code>REGISTER {username}</code>	Persistent connection; registers the client for routing
Client → Server	<code>PUBLISH {username} {orderId} {status}</code>	Short-lived connection; routes a status update to the named user
Server → Client	<code>ORDER_STATUS {orderId} {status}</code>	Delivered to the named user's persistent connection

Fields are pipe-delimited plain text. No authentication or encryption is applied to the socket protocol; the system is designed for localhost use only.

9. Testing

9.2 Test Cases

Category	Test Scenario	Input / Action	Expected Outcome
Authentication	Valid customer login	Username: malak, Password: 123456	Session set; Customer Dashboard loaded
Authentication	Wrong password	Correct username, wrong password	"Incorrect username or password." (red label)
Authentication	Unknown username	Non-existent username	"No account found with that username." (red label)
Authentication	Valid staff login — Admin	Username: admin, Password: admin123	Admin Dashboard loaded
Authentication	Valid staff login — Waiter	Username: ahmed, Password: waiter123	Waiter Dashboard loaded
Registration	Password too short	Password with 4 characters	"Password must be at least 6 characters long."
Registration	Phone wrong length	10-digit phone number	"Phone number must be exactly 11 digits."
Registration	Duplicate username	Username already in Database	"Username already exists. Please choose another."
Registration	Valid registration	All fields valid, unique username	Customer created with \$500 balance; session set
Reservation	Party exceeds capacity	Party size > table capacity	IllegalArgumentExcep ^{tion} caught; error feedback shown
Reservation	Exact-time double booking	Same table + same datetime as ACTIVE reservation	"Table is already reserved." feedback
Reservation	Past date	Date before today	isValidReservation() false; feedback shown
Reservation	Time outside range	Time before 10:00 or after 23:00	isValidReservation() false; feedback shown
Reservation	Valid cancellation	Cancel an ACTIVE reservation	Status → CANCELLED; table status → AVAILABLE
Ordering	Unavailable menu item	Add item where isAvailable() = false	Feedback label: item unavailable
Ordering	Valid add to order	Select available item and click Add	OrderItem added; running total updated

Category	Test Scenario	Input / Action	Expected Outcome
Payment	Insufficient balance	Total > customer.getBalance()	Error Alert: insufficient balance
Payment	Already-paid invoice	Re-attempt checkout on PAID order	Informational Alert; redirect
Payment	Valid payment	Sufficient balance; select method; confirm	Balance deducted; loyalty points awarded; order PAID; TCP PAID push sent
Admin CRUD	Duplicate table number	Add table with existing number	"Table number already exists." feedback
Admin CRUD	Non-positive capacity	Table capacity = 0 or negative	"Capacity must be a positive integer." feedback
Waiter	Place walk-in order	Leave customer field blank/null in take-order form	Order created with null customer reference